MSH chain on a 2D substrate

Function reference

`msh_substrate_tn.ipynb`	Julia, TensorBinding.jl / ITensors: quantics MPO, penalized DMRG
`msh_substrate_python.ipynb`	Python, SciPy / MUMPS: sparse matrix, dense and shift-invert reference
`msh_substrate_scaling.ipynb`	Python: cost of the methods against system size (plots)

Repository `MSH_Tensor_Network_Methods`, branch `msh-substrate`

30 October 2026

This document explains every function and struct defined in the three notebooks (folder `msh_substrate/`): what it computes, the method, its inputs and outputs, the numerical settings that matter, and where it is used. Numbers stated in the markdown of the notebooks are quoted as such; numbers from the outputs are those of the executed notebooks in the repository. Function names in `typewriter` are those of the code.

Contents

1 Overview **3**

1.1 Model and conventions . . . 3
1.2 Quantics encoding (Julia) . . . 5
1.3 Nambu bases of the two codes . . . 5
1.4 Workflow and data files . . . 5

2 Part I — Tensor-network notebook (`msh_substrate_tn.ipynb`) **7**

2.1 Section 1: setup and helpers . . . 7
2.2 Section 2: geometry . . . 8
2.3 Section 3: Hamiltonian MPO . . . 8
2.4 Section 4: check data for the Python notebook . . . 9
2.5 Section 5: MPO size and build time against system size . . . 9
2.6 Section 6: zero-mode pipeline . . . 10
2.7 Section 7: zero-mode benchmark . . . 12
2.8 Section 8: a substrate the zero modes leak into . . . 12

3 Part II — Python notebook (`msh_substrate_python.ipynb`) **14**

3.1 Setup . . . 14
3.2 Section 1: Hamiltonian . . . 14
3.3 Section 2: check against the tensor-network Hamiltonian . . . 15
3.4 Section 3: lowest levels against substrate width . . . 15
3.5 Section 4: sparse matrix size against system size . . . 15
3.6 Section 5: zero-mode benchmark, dense and sparse . . . 15
3.7 Section 6: MUMPS estimates beyond the benchmark . . . 16

3.8 Section 7: charge difference, mask method and single-mode formula 17
3.9 Section 8: a substrate the zero modes leak into 18

4 Part III — Scaling notebook (`msh_substrate_scaling.ipynb`) 20
4.1 Loading and helpers (cell 1) 20
4.2 Section 1: time 20
4.3 Section 2: memory 20
4.4 Section 3: accuracy 21
4.5 Section 4: bond dimensions 21
4.6 Section 5: summary and crossover 21
4.7 Section 6: a substrate the zero modes leak into 22

1 Overview

The three notebooks measure how the cost of computing the Majorana zero modes of an MSH chain grows when the chain sits on a 2D superconducting substrate that is made larger and larger, and compare a tensor-network method with dense and sparse-direct methods. The model is a $2^{L_x} \times 2^{L_y}$ square lattice with hopping, Rashba coupling and s-wave pairing on every site, and a centred magnetic chain of ℓ sites in the middle row (section 1.1). Two quantities are computed at every size: the splitting E_f of the zero-mode pair and its charge difference $\Delta Q = \langle \psi_f | \tau_z | \psi_f \rangle$.

- `msh_substrate_tn.ipynb` (Julia) builds the BdG Hamiltonian as a quantics MPO on $2 + L_y + L_x$ qubits, finds the near-zero subspace by penalized DMRG on $K = H^2 + \eta^2$, localizes the two Majoranas with the position operator and pairs them into ψ_f.
- `msh_substrate_python.ipynb` (Python) builds the same Hamiltonian as a sparse matrix and solves it by dense `eigh` and by shift-invert Lanczos with SuperLU or MUMPS; it also checks the Hamiltonian against the MPO, the convergence in the substrate size, the charge-difference formula, and scans the parameters for a point where the zero modes leak into the substrate.
- `msh_substrate_scaling.ipynb` (Python) reads the CSV files of the other two and compares time, memory, accuracy and bond dimensions, and finds the crossover between the methods.

Two parameter points are used: the benchmark point $\mu = -7$, $\Delta = 2.4$, $J = 5.8$, $\ell = 12$, where the substrate is a gapped trivial superconductor and the zero modes stay on the chain row, and the leaking point $\Delta = 0.6$, $\mu = -3$, $J = 3$, $\ell = 64$ (gap 0.090), where they spread over several rows.

Main results.

- *Benchmark point.* The bond dimensions do not grow with the system size: $\chi(H) = 7$, $\chi(K) \leq 38$, $\chi(\psi) \leq 26$ from one row to 2048×2048 sites. The TN time grows as $N^{0.14}$ (126 s at 1024^2, 140 s at 2048^2, 1.8 GB process memory), against $N^{1.5}$ for SuperLU and MUMPS (MUMPS: 70 s at 512^2, 1013 s and 28.6 GB at 1024^2; SuperLU runs out of memory there) and $N^{3.07}$ for dense diagonalization. At this point the MUMPS factors contain subnormal numbers, which are slow on x86; with flush-to-zero / denormals-are-zero (`mumps_ftz`) the same results take 43 s at 512^2 and 236 s at 1024^2. The crossover with the TN is at about 589^2 sites for MUMPS out of the box and at about 786^2 with flush-to-zero. TN errors: $\leq 2.5 \times 10^{-8}$ in ΔQ and $\leq 1.1 \times 10^{-11}$ in E_f.
- *Leaking point.* The bond dimensions are larger but still flat in the size ($\chi_\psi = 114$–137, $\chi_K = 31$–37), and the TN time is 1.3–3.0 thousand seconds from 256^2 to 2048^2. MUMPS is faster at every size up to 1024^2 (295 s) and about equal at 2048^2 (estimate 2326 s against 2556 s), where it needs ~121 GB against 2.3 GB for the TN. TN errors $\leq 2.6 \times 10^{-12}$ in E_f and $\leq 9 \times 10^{-10}$ in ΔQ.
- *DMRG details that matter.* K must be built with `cutoff_K = 1e-20` for ΔQ at the 10^{-9} level; the second DMRG state is started from the mirror image of the first, which fixes a convergence failure at 2048^2; and which basis of the zero-mode pair DMRG returns (Majoranas, $\chi \approx 114$, or two-ended mixtures, $\chi \approx 136$) varies with the random start, so the time of a size is uncertain by up to a factor 1.7.

1.1 Model and conventions

Lattice. A square substrate of $N_x \times N_y = 2^{L_x} \times 2^{L_y}$ sites with open boundaries in both directions; $N = N_x N_y$ sites and BdG dimension $D = 4N$. Sites are labelled by the 0-indexed column i_x and row i_y. The magnetic chain is the centred segment of ℓ sites in the middle row,

$$i_{y0} = \lfloor N_y/2 \rfloor, \qquad i_x = x_0, \ldots, x_0 + \ell - 1, \quad x_0 = \lfloor (N_x - \ell)/2 \rfloor$$

(`chain_row`, `chain_x` in Julia; `chain_sites` in Python). With $L_y = 0$ there is a single row and the system is the MSH chain of `msh_chain_tn.ipynb` with a centred topological segment.

Hamiltonian. The particle block is

$$H_0 = \sum_{\mathbf{r}} c_{\mathbf{r}}^\dagger(-\mu)c_{\mathbf{r}} + \sum_{\mathbf{r}}[c_{\mathbf{r}+\hat{x}}^\dagger h_x c_{\mathbf{r}} + c_{\mathbf{r}+\hat{y}}^\dagger h_y c_{\mathbf{r}} + \text{h.c.}] + J\sum_{\mathbf{r}\in\text{chain}} c_{\mathbf{r}}^\dagger\,\mathbf{S}(\mathbf{r})\cdot\sigma\, c_{\mathbf{r}},$$
$$h_x = -t + i\alpha\sigma_y, \qquad h_y = -t - i\alpha\sigma_x,$$

with s-wave singlet pairing $\Delta\, i\sigma_y$ on every site; in the Julia Nambu basis

$$H_{\text{BdG}} = \begin{pmatrix} H_0 & \Delta\, i\sigma_y \\ (\Delta\, i\sigma_y)^\dagger & -H_0^* \end{pmatrix}.$$

Two spin textures are used: `S_z`, all moments along $\hat{z}$ (topological; every benchmark), and `S_tilt`, $\mathbf{S}(i_x) = (\sin 0.3 i_x \cos 0.7,\ \sin 0.3 i_x \sin 0.7,\ \cos 0.3 i_x)$, a generic texture with all three components, used only to check the Hamiltonian. The texture depends on i_x only, since the chain lies in one row.

Parameter points.

Symbol	Benchmark point	Second point	Code name (Julia / Python)	Meaning
t	1	1	`t` / `t`	nearest-neighbour hopping
μ	-7	-3	`μ` / `mu`	chemical potential
α	0.9	0.9	`α` / `alpha`	Rashba spin–orbit coupling
Δ	2.4	0.6	`Δ` / `Delta`	s-wave pairing
J	5.8	3	`J` / `J`	exchange coupling to the moments
ℓ	12	64	`ℓ` / `ell`	chain length (zero-mode runs)
used in	TN 7, Py 5–7	TN 8, Py 8		

The size scans without eigensolver use $\ell = 48$ (width scan, and Python section 3) and $\ell = 3N_x/4$ (square scan).

The markdown describes the two points as follows. At the benchmark point the bare band (± 4.74 with the Rashba term) is empty, so the substrate is a gapped, trivial superconductor on which the chain stays topological; the zero modes barely enter it (gap ≈ 0.55, decay within one row; only $\sim 2\times10^{-5}$ of the weight lies four or more rows away), which favours the tensor network. With $\ell = 12$ the pair splitting $E_f \approx 4\times10^{-4}$ and the charge difference $|\Delta Q| \approx 3\times10^{-3}$ are far above round-off, so they measure accuracy ($\ell = 48$ would give a splitting $\sim 10^{-14}$). The second point, found by the $\mu \times J \times \Delta$ scan of Python section 8, is inside the band, topological and gapped (gap 0.0902), and its zero mode spreads over several rows: 0.32 of its weight on the chain row, 0.13 four or more rows away, decaying as $e^{-|\Delta y|/2}$. The splitting decays along the chain with a length of about 7 sites; $\ell = 64$ gives $E_f/E_{\text{gap}} = 10^{-4}$, comparable with the benchmark point. The converged exact values quoted there are $E_f = 9.04771\times10^{-6}$ and $\Delta Q = 4.58344\times10^{-4}$.

Quantities. Every zero-mode run returns three numbers: the pair energy $E_f > 0$ of the fermion $\psi_f = (\gamma_1 + i\gamma_2)/\sqrt{2}$ built from the two Majoranas, the charge difference

$$\Delta Q = \langle\psi_f|\tau_z|\psi_f\rangle = \sum_{\mathbf{r},\sigma}(|u_\sigma(\mathbf{r})|^2 - |v_\sigma(\mathbf{r})|^2),$$

which equals the charge difference of the two logical states (Python section 7), and the gap E_{gap}, the next level above the pair.

1.2 Quantics encoding (Julia)

A single-particle BdG vector is an MPS, and H an MPO, on $L_x+L_y+2 = \log_2 N+2$ sites of dimension 2:

$$[\,\text{nambu};\ \text{spin};\ y_1,\dots,y_{L_y};\ x_1,\dots,x_{L_x}\,],$$

most significant bit first. The column and row are $i_x = \sum_{k=1}^{L_x} b_k^x 2^{L_x-k}$ and $i_y = \sum_{k=1}^{L_y} b_k^y 2^{L_y-k}$, so the position register (y bits, then x bits) read as one binary number is

$$n = i_x + i_y N_x,$$

TensorBinding's 2D row-major convention. A shift by one site in x is therefore $|n\rangle \to |n+1\rangle$ and a shift by one row is $|n\rangle \to |n+N_x\rangle$, both binary adders on the whole register. In the dense form of the MPO (`dense`, big-endian over the sites) the index is $n_b \cdot 2N + \sigma N + n$ with $n_b, \sigma \in \{0,1\}$. The x qubits are the sites $3+L_y,\dots,2+L_y+L_x$ (`xqubits`).

1.3 Nambu bases of the two codes

	Julia (TensorBinding)	Python (kwant convention)
Spinor per site	$(c_\uparrow, c_\downarrow, c_\uparrow^\dagger, c_\downarrow^\dagger)$	$(c_\uparrow, c_\downarrow, c_\downarrow^\dagger, -c_\uparrow^\dagger)$
Matrix index	$n_b \cdot 2N + \sigma N + n$	$4n + c$
On-site term	$-\mu + J\mathbf{S}\cdot\sigma$; pairing $\Delta\, i\sigma_y$	$-\mu\tau_z + \Delta\tau_x + J\,\tau_0\mathbf{S}\cdot\sigma$
Hopping $H_{\mathbf{r}+\hat{d},\mathbf{r}}$ $(\hat{x},\hat{y})$	$-t + i\alpha\sigma_y$, $-t - i\alpha\sigma_x$	$-t\tau_z + i\alpha\tau_z\sigma_y$, $-t\tau_z - i\alpha\tau_z\sigma_x$
Particle–hole operator	$C = \tau_x K$ (`Cconj`)	$C = \tau_y\sigma_y K$
Charge operator	τ_z on the Nambu site (`tz_on`)	$+1$ on $c = 0,1$, -1 on $c = 2,3$

Per site, the Python components (a,b,c,d) are $(u_\uparrow, u_\downarrow, v_\downarrow, -v_\uparrow)$ in terms of the Julia components $(u_\uparrow, u_\downarrow, v_\uparrow, v_\downarrow)$. The signed permutation `julia_to_python` (Python section 2) implements this map; τ_z and therefore ΔQ are the same in both bases.

1.4 Workflow and data files

Run order (all paths relative to `msh_substrate/`):

`msh_substrate_tn.ipynb` → `msh_substrate_python.ipynb` → `msh_substrate_scaling.ipynb`.

Python section 2 needs `data/tn_check_H.npz` from the TN notebook; Python section 8c needs `py_mumps_estimates.csv` from Python section 6; the scaling notebook only loads files. All timings are single-threaded (`BLAS.set_num_threads(1)` in Julia; `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS` set to 1 before NumPy is imported), except the parameter scan of Python section 8a, which uses 8 processes. The markdown gives about 2.5 hours for TN section 8; from the recorded times, TN section 7 takes about 12 minutes and section 8 about 2.5 hours (8924 s); Python section 5 takes at least 40 minutes (dense 64×64: 886 s, MUMPS 1024^2: 1013 s) and section 8 several minutes (scan 198 s, MUMPS 1024^2: 295 s); the scaling notebook runs in seconds.

File	Written by	Content
`data/tn_check_H.npz`	TN sec. 4	ten dense MPO matrices, keys `H_Lx_Ly_ℓ_{z,tilt}` (not committed)
`data/zm_progress.log`	TN sec. 7	one line per finished size of the benchmark
`tn_zero_modes.csv`	TN sec. 7	TN benchmark, 22 columns (committed)
`tn_candidate.csv`	TN sec. 8	TN at the second point, 24 columns (committed, resumable)
`py_zero_modes.csv`	Py sec. 5	dense / SuperLU / MUMPS / MUMPS with FTZ/DAZ benchmark (resumable)
`py_mumps_estimates.csv`	Py sec. 6	MUMPS analysis estimates, $L = 7 \dots 11$
`py_mu_scan.csv`	Py sec. 8a	1305 points of the $\mu \times J \times \Delta$ scan
`py_candidate.csv`	Py sec. 8c	MUMPS at the second point (resumable)

Mapping between the codes.

Quantity	Julia (`msh_substrate_tn`)	Python (`msh_substrate_python`)
Geometry	`Substrate`, `Nx`, `Ny`, `chain_row`, `chain_x`	`chain_sites`
Texture	`S_z`, `S_tilt`	`S_z`, `S_tilt`
H	`msh_substrate` (MPO)	`ham` (CSR)
Check of H	section 4 (export)	`julia_to_python`, section 2
Size of H	`mpo_size`, section 5	`size_info`, section 4
Zero modes	`zero_mode_subspace`, `fermion_from_subspace`	`solve`, `pair_values`
ΔQ	`ΔQ_mode`	`pair_values`; `mask_check` (section 7)
Benchmark	`zm_run` (section 7)	`solve` in `in_fresh_process` (section 5)
Second point	`cand_run` (section 8)	`solve_candidate` (section 8c)

2 Part I — Tensor-network notebook (`msh_substrate_tn.ipynb`)

The Hamiltonian is a quantics MPO on $\log_2 N + 2$ sites; the zero modes come from penalized DMRG on $K = H^2 + \eta^2$. Dense matrices appear only in section 4, to export check data for the Python notebook. Packages: `TensorBinding` (alias `TB`), `ITensors`, `ITensorMPS`, `LinearAlgebra`, `Printf`, `NPZ`, `Random`, `Logging`, `DelimitedFiles`.

2.1 Section 1: setup and helpers

Global settings: `BLAS.set_num_threads(1)` (single thread, as in the Python notebook, so that timings compare like with like); the Pauli matrices `σ0, σx, σy, σz` (`ComplexF64`); `DATA_DIR = "data"`, created if missing, for the check data.

`diag_mpo(ψ, pos)`

```
δ = ITensor(ComplexF64, s, s', dag(s''))
for j in 1:2; δ[s => j, s' => j, s'' => j] = 1; end
W[k] = replaceprime(ψ[k] * δ, 2 => 0; tags=tags(s))
```

Turns an MPS ψ on the position qubits `pos` into the diagonal MPO $\mathrm{diag}(\psi) = \sum_n \psi_n |n\rangle\langle n|$. Each site tensor is contracted with the copy tensor $\delta_{ss's''}$, which places the value $\psi[k]_{s=j}$ on the diagonal $(s', s) = (j, j)$; the `replaceprime` renames s'' back to s. The bond dimension of the MPO equals that of ψ, and the operation is exact. Used for the x-bond mask and the exchange in `msh_substrate`, and by `diag_mpo_exact`.

`vector_mps(v, q; cutoff=1e-14)`

MPS of a vector v of length $2^{|q|}$ on the qubits `q` (`q[1]` the most significant bit). The vector is reshaped into a $|q|$-index array, the indices reversed with `permutedims` (Julia's column-major reshape puts the least significant bit first), and compressed by TT-SVD (`MPS(A, q; cutoff)`) with truncation `cutoff = 1e-14`, i.e. exact up to round-off. `@disable_warn_order` suppresses ITensors' warning about tensors with many indices. Cost and memory are $O(2^{|q|})$, since the full array is built.

`diag_mpo_exact(v, pos; cutoff=1e-14)`

`diag_mpo(vector_mps(v, pos; cutoff), pos)`: the exact diagonal MPO $\sum_n v_n |n\rangle\langle n|$ of a full-length vector. Not used by the current Hamiltonian; an earlier version compressed length-N vectors with it, which, according to the markdown of section 3, took 12–14 s of the build at 512×512 and 1024×1024 sites. It is kept as the reference construction.

`row_profile_mps(yvecs, w, ypos, xpos; cutoff=1e-14)`

```
ψx = vector_mps(w, xpos; cutoff)
isempty(ypos) && return ψx
link = [Index(1, "Link,ly=$k") for k in eachindex(ypos)]
...  t = ITensor(ComplexF64.(yvecs[k]), s) * onehot(link[k] => 1)
ψx[1] *= onehot(link[end] => 1)
return MPS(vcat(T, collect(ψx)))
```

The MPS of $(\mathbf{y}_1 \otimes \cdots \otimes \mathbf{y}_{L_y}) \otimes w$ on $[\, y_1..y_{L_y};\ x_1..x_{L_x} \,]$: a product of two-component vectors $\mathbf{y}_k$ on the y qubits times a profile w of length N_x on the x qubits. Only w is compressed (`vector_mps`); each y qubit gets a single tensor joined to its neighbours by dimension-1 links, and the last link is attached to the first x tensor. The cost is $O(N_x)$ instead of $O(N)$. Two choices of `yvecs` are used: the

bits of one row i_y (each $\mathbf{y}_k = (1,0)$ or $(0,1)$), which gives a vector that is $w(i_x)$ on that row and zero elsewhere; and $\mathbf{y}_k = (1,1)$ for all k, which gives the same profile w on every row. With $L_y = 0$ it returns ψ_x unchanged. This is what makes the build of H cost $O(N_x)$ (section 3).

`dense(mpo, sites)`, `adjoint_mpo(K)`

`dense` contracts the whole MPO (`prod`) and reshapes it into a $D \times D$ matrix, big-endian over `sites` (first site most significant), giving the index $n_b \cdot 2N + \sigma N + n$. Small systems only (section 4). `adjoint_mpo(K) = swapprime(dag(K), 0, 1)` is the Hermitian conjugate of an MPO: complex conjugation of the tensors and exchange of bra and ket indices.

2.2 Section 2: geometry

`struct Substrate; Nx, Ny, nsites, chain_row, chain_x`

```
struct Substrate
    Lx::Int; Ly::Int; ℓ::Int
end
chain_row(g::Substrate) = Ny(g) ÷ 2
chain_x(g::Substrate) = ((Nx(g) - g.ℓ) ÷ 2):((Nx(g) - g.ℓ) ÷ 2 + g.ℓ - 1)
```

`Substrate(Lx, Ly, ℓ)` holds the geometry: $N_x = 2^{L_x}$ columns (`Nx`), $N_y = 2^{L_y}$ rows (`Ny`), N sites (`nsites`) and a chain of ℓ sites. `chain_row` is the chain row $\lfloor N_y/2 \rfloor$ (0 for one row), `chain_x` the 0-indexed range of chain columns. For even ℓ (all zero-mode runs) the chain is exactly centred, $x \to N_x - 1 - x$ maps it onto itself; for odd ℓ (one check case) it is off by half a site.

`S_z(ix)`, `S_tilt(ix)`

Spin textures as functions of the column: `S_z` returns $(0,0,1)$; `S_tilt` returns $(\sin 0.3 i_x \cos 0.7,\ \sin 0.3 i_x \sin 0.7,\ \cos 0.3 i_x)$, so that all three exchange components are nonzero and vary along the chain. Identical lambdas exist in the Python notebook.

2.3 Section 3: Hamiltonian MPO

`msh_substrate(g, S; t=1.0, μ=-7.0, α=0.9, Δ=2.4, J=5.8, cutoff=1e-13, sites=nothing)`

Returns `(HB, [ns; ss; pos])`: the BdG MPO of the chain with texture `S(ix)` on substrate `g`, and its site list. `ns` and `ss` are TensorBinding's Nambu and spin indices (`TB.nambu_index()`, `TB.spin_index()`), `pos` the $L_x + L_y$ qubits (`siteinds("Qubit", ...)`), split into `ypos` (first L_y) and `xpos`. If `sites` is given its indices are reused. `P(op, M) = TB.prepend_spin(M, ss, op)` puts a 2×2 spin matrix in front of a position MPO.

Hopping. The x bonds and the y bonds are built from TensorBinding's binary-adder MPO `TB.shift_pair_mpos(pos, q; cyclic=false)`, whose first element maps $|n\rangle \to |n+q\rangle$ on the whole register (bond dimension 2: the carry). With `cyclic=false` a carry out of the most significant bit is dropped, so states with $n + q \geq N$ are mapped to zero.

- **x bonds:** $K_x = K_1 \cdot \text{mask}$, with $K_1 =$ shift by 1 and

$$\text{mask} = \text{diag}((1,1)^{\otimes L_y} \otimes m), \qquad m(i_x) = \begin{cases} 0 & i_x = N_x - 1 \\ 1 & \text{otherwise.} \end{cases}$$

Without the mask the shift would join the last column of row i_y to the first column of row $i_y + 1$. The mask is built by `row_profile_mps(every_row, m, ypos, xpos)` and `diag_mpo`, the product by `apply(K1, mask; cutoff)` (the mask acts first). Spin matrix $h_x = -t\sigma_0 + i\alpha\sigma_y$.

- **y bonds:** K_y = shift by N_x (`shift_pair_mpos(pos, nx)`), which adds 1 to i_y and maps the top row to zero (open boundary); $h_y = -t\sigma_0 - i\alpha\sigma_x$. Only added when $L_y > 0$: with one row, $N_x = 2^L$ has no bit inside the register, the adder keeps only the low bits of q and would act as the identity.

For each bond the terms $h \otimes K$ and $h^\dagger \otimes K^\dagger$ (`P(Matrix(hop'), adjoint_mpo(K))`) are added, i.e. $c^\dagger_{\mathbf{r}+\hat{d}} h c_{\mathbf{r}}$ and its Hermitian conjugate.

Exchange. For each spin component k the vector $w(i_x) = JS_k(i_x)$ on the chain columns (zero elsewhere) becomes `diag_mpo(row_profile_mps(row(iy0), w, ypos, xpos), pos)`: nonzero on the chain row only, where `row(iy)` lists the bits of i_y as unit vectors, most significant first. The term $\sigma_k \otimes (\cdot)$ is added only when $\max|w| > 10^{-14}$, so `S_z` gives a single exchange term.

Assembly. $H_0 = -\mu\,\sigma_0 \otimes \mathbb{1} + \sum_{\text{bonds}} + \sum_k$ exchange, summed with `+(terms...; cutoff)` at `cutoff = 1e-13`. The pairing MPO is `Hp = P(:iSy, Δ * Id)` $= \Delta\, i\sigma_y \otimes \mathbb{1}$. The BdG MPO is

$$H_B = \tau_{pp} \otimes H_0 + \tau_{hh} \otimes (-H_0^*) + \tau_+ \otimes H_p + \tau_- \otimes H_p^\dagger,$$

with `TB.prepend_nambu` and the operators `:Pp`, `:Ph`, `:tp` $= |p\rangle\langle h|$, `:tm` $= |h\rangle\langle p|$; $-H_0^*$ is `-1.0 * dag(H0)` (`dag` conjugates the elements) and $H_p^\dagger$ is `adjoint_mpo(Hp)`. Again summed at `cutoff = 1e-13`.

Cost. Every diagonal is a product state on the y qubits times a compressed vector of length N_x, and the shifts are exact bond-dimension-2 MPOs, so the build costs $O(N_x)$ (markdown: milliseconds at every size). The resulting $\chi(H)$ is $\chi(H) = 7$ for spins along $\hat{z}$ at every size, from one row to 2048×2048 sites, and 8–9 for the tilted texture of the check cases; it does not depend on L_y.

Used in: section 4 (both textures), section 5 (build scan), `zm_run` and `cand_run` (with `PC...` for the second point).

2.4 Section 4: check data for the Python notebook

For the five geometries `check_cases` $= (L_x, L_y, \ell) = (3,0,6), (3,1,6), (3,2,5), (2,3,3), (4,2,10)$ (one row; two to eight rows; a substrate taller than wide) and both textures (`"z"`, `"tilt"`), the cell builds `msh_substrate`, converts it with `dense`, and stores the matrix under the key `H_$(Lx)_$(Ly)_$(ℓ)_$name` in a dictionary written by `npzwrite` to `data/tn_check_H.npz`. Dimensions $D = 32$ to 256. For each case it prints D, $\chi(H)$ and the Hermiticity error $\|H - H^\dagger\|$ ($\chi(H) = 7$ for `S_z`, 8–9 for `S_tilt`; $\|H - H^\dagger\| = 1.4 \times 10^{-13}$ to 9.0×10^{-13}). The comparison itself is done in Python section 2.

2.5 Section 5: MPO size and build time against system size

`mpo_size(H)`

`sum(length(storage(T)) for T in H)`: the number of stored elements of the MPO tensors, the memory measure compared with the number of nonzeros of the sparse matrix (Python section 4). Also used for K in `zm_run` and `cand_run`.

Build scan. `scan` is the width scan $(6, L_y, 48)$, $L_y = 0 \ldots 6$, followed by the square scan $(L, L, 3{\cdot}2^L/4)$, $L = 4 \ldots 11$ (up to 2048×2048 sites). For each size the cell builds H once as a warm-up (so compilation is not timed) and then times a second build with `@elapsed`; it collects $(L_x, L_y, \ell, N, \chi, \text{mem}, t_{\text{build}})$ in `build` (not written to a file) and prints N, D, $\chi(H)$, the MPO elements and the build time. At 2048×2048: $\chi(H) = 7$, 3116 MPO elements, build 0.04 s; all builds take 0.01–0.04 s, and the number of elements grows linearly in the number of qubits (612 for one row of 64 sites).

2.6 Section 6: zero-mode pipeline

The pipeline of `msh_chain_tn.ipynb` (sections 8–9) adapted to the substrate: penalized DMRG for the near-zero subspace, $\hat{X}$ localization and pairing, then ΔQ.

`quietly(f)`

Runs the closure `f` with a logger that shows errors only. KrylovKit warns ("ignoring imaginary component") at every DMRG step on these complex MPOs; the comment states that the imaginary parts are round-off. Wraps the DMRG calls of `zero_mode_subspace`.

`norm_Hψ(H, ψ)`

$\|H\psi\|$, from the exact product `apply(H, ψ; alg="naive", truncate=false)`, orthogonalized to site 1 so that the norm is the norm of the first tensor. For a normalized eigenvector this is $|E|$; for an approximate one it is an upper bound on the distance of the nearest eigenvalue from zero. Gives `E_sub` and `E_gap`.

`apply_site(op, ψ, j)`, `τz_on(ψ)`, `ΔQ_mode(ψ)`

`apply_site` applies a single-site operator `op` (indices s', s) to site j of a copy of ψ; bonds are unchanged. `τz_on` does the same with $\tau_z = \mathrm{diag}(1, -1)$ on the Nambu site (site 1). `ΔQ_mode(ψ) = real(inner(ψ, τz_on(ψ))) = ⟨ψ|τ_z|ψ⟩`, the charge difference of a normalized fermion mode.

`mps_size(ψ)`, `xqubits(g)`

`mps_size`: stored elements of the MPS tensors. `xqubits(g) = (3 + g.Ly):(2 + g.Ly + g.Lx)`: site numbers of the x qubits, most significant first (after Nambu, spin and the L_y y qubits).

`Cconj(ψ)`

Particle–hole conjugate $C\psi = \tau_x K\psi$ of an MPS: `dag(ψ)` conjugates all tensors, then τ_x swaps the particle and hole components on the Nambu site. The near-zero subspace is closed under C.

`mirror_x(ψ, sites, xq)`

The mirror image $x \to N_x - 1 - x$ of an MPS. In the quantics encoding $N_x - 1 - i_x$ is the bitwise complement of i_x, so the mirror is a NOT gate $\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$ on every x qubit (site numbers `xq`); it is a product operator and leaves the bond dimensions unchanged. For a centred chain (even ℓ) it maps one chain end onto the other. Rows and the Nambu and spin components are untouched.

`zero_mode_subspace(HB, sites; nzero=2, η=1e-2, seed=1234, nsweeps=20, ..., xq=nothing)`

Penalized DMRG for `nzero` near-zero states and one more state as a gap check. Steps:

1. $K = H^2 + \eta^2$, $\eta = 10^{-2}$, by `TB.build_K(HB, sites, 0.0, η; cutoff_K)`: $H \cdot H$ by `apply` with truncation `cutoff_K` and TensorBinding's default cap `maxdim_K = 200`, plus $\eta^2 \mathbb{1}$. Timed as `t_K`.
2. `Random.seed!(seed)`; for $k = 1, \ldots,$`nzero`$+1$ a DMRG run on K from `random_mps(sites; linkdims=10)`. The first state is the ground state of K, i.e. the eigenstate of H closest to zero; later states use `dmrg(K, ψs, ψ0; weight)`, which adds the penalty $w \sum_j |\psi_j\rangle\langle\psi_j|$ ($w =$`weight`$= 10$) for the states already found. All three runs are timed together as `t_dmrg`.
3. Each state is normalized. The last one is the gap state ψ_{gap}; the first `nzero` span the zero-mode subspace.

DMRG settings: `nsweeps = 20`, `maxdim = [10, 20, 50, 100, 200]` (the last value repeats), `cutoff = 1e-14`, `noise = [1e-6, 1e-7, 1e-8, 0.0]`, `eigsolve_krylovdim = 20`, `eigsolve_maxiter = 3`, `outputlevel = 0`.

Truncation of K (`cutoff_K = 1e-20`). The markdown explains the choice: ΔQ is first order in any admixture of bulk states in the pair (energies only second order), so it is the most sensitive quantity. With `cutoff_K = 1e-14` the one-row chain had $|\Delta Q_{\mathrm{TN}} - \Delta Q_{\mathrm{exact}}| = 1.3\times10^{-5}$, from a weight 4.7×10^{-10} outside the exact pair that did not depend on sweeps or the DMRG cutoff; with `1e-20` the error was 1.3×10^{-10}. With a substrate the error was a few $\times10^{-9}$ for either value, limited by DMRG convergence. The tighter cutoff doubles $\chi(K)$ (18 to 38) and makes DMRG 1.2–1.6 times slower on the large substrates. The inverse-iteration refinement of `msh_chain_tn.ipynb` is not needed ($E_f \gg 2 \times 10^{-6}$) and was not an alternative: with E_f close to its shift $\sigma = 10^{-3}$ it made $L_y = 1$ worse (5.7×10^{-5}).

Start of the second state. With a random start the second DMRG can end in a wrong state: the markdown reports that at the second parameter point and 2048×2048 sites it stopped at $\|H\psi_2\| = 0.60$ (a substrate band-edge state, bond dimension at the cap of 200), and the gap check then picked up the missing zero mode. The second state is therefore started from a state built from ψ_1, after removing its ψ_1 component (`orth(φ) = +(φ, -inner(ψs[1], φ) * ψs[1]; cutoff=1e-14)`), in this order:

- if `xq` is given, the mirror image `mirror_x(ψ1, sites, xq)`, if its orthogonal part has norm > 0.1. When ψ_1 is a Majorana at one chain end this is the Majorana at the other end. Orthogonalized, it keeps norm 0.98–1.00;
- otherwise $C\psi_1$ (`Cconj`), if its orthogonal part has norm > 0.1 (the subspace is closed under C). It is useless when ψ_1 is a single Majorana: then $C\psi_1 \propto \psi_1$ and the orthogonal part has norm 0.000;
- otherwise the random MPS (`start2 = "random"`).

The random MPS of state 2 is drawn in any case, so the random stream of state 3 does not depend on the choice. Both norms are computed and returned whichever start is taken; `C_perp` ≈ 0 means that ψ_1 came out a single Majorana.

Which basis of the pair DMRG returns. The near-zero subspace is two-dimensional, and DMRG can return any orthonormal basis of it: two localized Majoranas, or two-ended mixtures like ψ_f. For the exact modes at the second parameter point (TT-SVD with cutoff 10^{-14}) a Majorana needs $\chi = 121$ and ψ_f needs 139, independent of the system size and of the position of the chain on the binary grid. Which basis comes out is set by the random start of ψ_1 and varies from size to size (section 8: $\chi \approx 114$ where ψ_1 is a Majorana, ≈ 136 where it is a mixture), so the DMRG time of a size is uncertain by up to $(137/114)^3 \approx 1.7$. Starting ψ_1 from a random MPS projected onto the left half of the substrate was tried and did not fix this at 512^2 and above (markdown).

Returns the named tuple `(ψs, E_sub, E_gap, K, t_K, t_dmrg, start2)`: the `nzero` states, their $\|H\psi\|$, the gap estimate $E_{\mathrm{gap}} = \|H\psi_{\mathrm{gap}}\|$ (which should sit at the bulk gap), K, the two times, and `start2 = (kind, C_perp, mirror_perp)` with `kind` ∈ {`"Cψ1"`, `"mirror"`, `"random"`} and the norms of the two orthogonalized candidates (`NaN` where not computed).

`fermion_from_subspace(HB, sites, ψs, g; cutoff=1e-14)`

Builds the Majoranas and the fermion mode from the two subspace states.

1. Overlap matrix $S_{ab} = \langle\psi_a|\psi_b\rangle$ and the position matrix $X_{ab} = \langle\psi_a|\hat{X}|\psi_b\rangle$ with $\hat{X} = \sum_k 2^{L_x-k}\hat{n}_k$ on the x qubits only ($\hat{n}_k = |1\rangle\langle1|$ on qubit k, applied by `apply_site`); $\hat{X}$ is the column i_x, independent of the row.
2. The generalized Hermitian eigenproblem $Xc = x\,Sc$ (`eigen(Hermitian(X), Hermitian(S))`, both symmetrized) gives the Majorana centres $x_1 \le x_2$ (0-indexed columns) and the coefficients; `combine` forms $\gamma_i = c_1\psi_1 + c_2\psi_2$ (sum at `cutoff`) and normalizes it.
3. Phase fix: if $C\gamma = e^{i\theta}\gamma$ then $e^{i\theta/2}\gamma$ is self-conjugate; $\theta = \arg\langle\gamma|C\gamma\rangle$.
4. $\psi_f = (\gamma_1 + i\gamma_2)/\sqrt{2}$, normalized; $E_f = \mathrm{Re}\,\langle\psi_f|H|\psi_f\rangle$. If $E_f < 0$, $\psi_f \to C\psi_f$ and $E_f \to -E_f$, so ψ_f is always the $+E$ member.

Returns `(centers, γs, ψf, E_f, C)`. The function computes its own x-qubit range and its own C (the same as `xqubits` and `Cconj`).

2.7 Section 7: zero-mode benchmark

`RUN_ZM = true`; `zm_file = "tn_zero_modes.csv"`; `zm_cols`, the 22 column names. `zm_scan` is the width scan $(6, L_y)$, $L_y = 0 \dots 6$, and the square scan (L, L), $L = 4, 5, 7, \dots, 11$ ($L = 6$ is in the width scan), 14 sizes sorted by $(L_x + L_y, L_y)$, i.e. increasing N. All with $\ell = 12$ and `S_z`.

`zm_run(Lx, Ly; ℓ=12)`

One benchmark size: `msh_substrate` (timed as `t_H`), `zero_mode_subspace` with `xq = xqubits(g)`, `fermion_from_subspace` (timed as `t_pair`) and `ΔQ_mode`. Returns the row

```
Lx, Ly, ell, N, D, t_H, t_K, t_dmrg, t_pair, t_total, chi_H, chi_K, chi_psi, mem_K_MB,
                  mem_psi_MB, rss_MB, E_f, dQ, E_gap, x1, x2, start2
```

with `t_total` the sum of the four times; χ the largest bond dimensions (`maxlinkdim`; for ψ the largest of the two subspace states); memories as stored elements ×16 bytes in MB (10^6 bytes), for K and the sum over the two subspace states; `rss_MB = Sys.maxrss()/1e6`, the peak resident memory of the whole Julia process so far (it includes the Julia runtime and compiled code, and, since N increases along the scan, it is the peak up to and including this size); `start2` only the kind of start.

Driver and file logic. If `RUN_ZM = false` and the CSV exists, it is loaded with `readdlm(...; header=true)`. Otherwise `zm_run(4, 0)` is run once as a warm-up (compilation), then the scan runs with one line per finished size appended to `data/zm_progress.log` (flushed after each size); the CSV is written only at the end, header plus `writedlm` of all rows. This run is not resumable: an interrupted scan leaves only the log. The cell prints one line per size (times, χ_H, χ_K, χ_ψ, $K + \psi$ memory, RSS, E_f, ΔQ, gap, start of ψ_2).

Results: $\chi(K) = 31$–38 and $\chi(\psi) = 7$–26 over the whole scan, with no growth along the square scan; `t_total` = 126 s and 140 s, RSS 1841 MB at both; K + MPS 1.40 MB at 2048×2048; the mirror start at every size; at 2048×2048: $E_f = 3.712285 \times 10^{-4}$, $\Delta Q = -3.446600 \times 10^{-3}$, gap 0.5529, the converged values from $L = 6$ on.

2.8 Section 8: a substrate the zero modes leak into

`RUN_CAND = true`; `cand_file = "tn_candidate.csv"`; `const PC = (μ = -3.0, Δ = 0.6, J = 3.0)` (t and α keep their defaults); `cand_scan` = $(7, 6)$, $(8, 8)$, $(9, 9)$, $(10, 10)$, $(11, 11)$, i.e. 128×64 and 256^2 to 2048^2 sites; $\ell = 64$, `S_z`.

`cand_run(Lx, Ly; ℓ=64)`

As `zm_run`, with `msh_substrate(g, S_z; PC...)`. Returns the 24 columns

```
Lx, Ly, ell, N, t_H, t_K, t_dmrg, t_pair, t_total, chi_K, chi_psi, mem_K_MB, mem_psi_MB, rss_MB,
              E_f, dQ, E_gap, E_sub1, E_sub2, x1, x2, start2, C_perp, mirror_perp
```

i.e. in addition $\|H\psi_1\|$, $\|H\psi_2\|$ (the check that the second DMRG state is a zero mode) and the full `start2` tuple with the norms of the orthogonalized $C\psi_1$ and mirror image; without `D` and `chi_H`. `rss_MB` is the process peak so far, so it includes section 7.

Resume logic. `done` is the set of (L_x, L_y) already in the CSV. If `RUN_CAND`, the header is written when `done` is empty, and each missing size is run and appended as one line as soon as it finishes (`open(..., "a")`), with a progress line `done Nx×Ny:  t s`. An interrupted run therefore resumes at the next size. Finally the file is read and printed: substrate, N, total time, χ_K, χ_ψ, RSS, E_f, ΔQ, gap, $\|H\psi_2\|$, $x_1 \dots x_2$, and the start with its two norms. The markdown gives a run time of about 2.5 hours. Results (mirror start at every size; mirror_perp = 1.000; $\|H\psi_2\| = 9.06$–9.08×10^{-6}, i.e. E_f):

substrate	total	χ_K	χ_ψ	C_perp
128×64	516 s	31	115	0.002
256^2	1340 s	31	137	0.386
512^2	1536 s	35	114	0.000
1024^2	2976 s	34	135	0.317
2048^2	2556 s	37	114	0.000

χ_ψ is 114–115 where ψ_1 came out a single Majorana (C_perp ≈ 0) and 135–137 where it is a mixture; this is why 1024^2 takes longer than 2048^2. At 2048×2048: $E_f = 9.047709{\times}10^{-6}$, $\Delta Q = 4.583431{\times}10^{-4}$, against the exact 9.047709×10^{-6} and 4.583435×10^{-4}.

3 Part II — Python notebook (`msh_substrate_python.ipynb`)

The same model as a sparse CSR matrix (SciPy), with dense diagonalization where it fits and shift-invert Lanczos with a sparse direct solver beyond. Packages: NumPy, `scipy.linalg`, `scipy.sparse`, `scipy.sparse.linalg` (`eigsh`, `splu`, `LinearOperator`), `mumps` (python-mumps, MUMPS 5.8 sequential, conda-forge), `multiprocessing`, `resource`, matplotlib.

3.1 Setup

The thread variables are set to 1 before NumPy is imported. `DATA_DIR = "data"`. Pauli matrices `s0, sx, sy, sz` and the Kronecker products (first factor Nambu τ, second spin σ) `tzs0` $= \tau_z\sigma_0$, `txs0`, `t0sx`, `t0sy`, `t0sz`, `tzsx`, `tzsy`. `S_z` and `S_tilt` are the same textures as in Julia.

`in_fresh_process(fn, *args)`

```
ctx = mp.get_context("fork"); q = ctx.Queue()
def target():
    try:
        out, status = fn(*args), "ok"
    except BaseException as e:
        out, status = None, f"{type(e).__name__}: {e}".replace(",", ";").strip()
    q.put((out, resource.getrusage(resource.RUSAGE_SELF).ru_maxrss / 1024, status))
```

Runs `fn(*args)` in a forked child process and returns `(result or None, peak RSS in MB, status)`. The peak is the child's `ru_maxrss` (kB on Linux, divided by 1024). Fork is used so that `fn` may be a closure or lambda (nothing is pickled except the result). Every calculation that builds a large matrix runs this way, for two reasons: the kernel stays small, and every measurement starts from the same baseline (a forked child inherits the kernel's resident memory, about 110 MB per the markdown, which would otherwise grow from one calculation to the next). Errors in the child (including `MemoryError`) become the status string `"Type: message"`, with commas replaced by semicolons so it can go into a CSV. The parent polls the queue with a 5 s timeout and checks `p.is_alive()`, so a child that dies without reporting (e.g. killed by the out-of-memory killer) gives `(None, nan, "process died (exit code ...)")` instead of hanging.

3.2 Section 1: Hamiltonian

`shift(n)`

$n \times n$ CSR matrix with ones on the first subdiagonal, $|i\rangle \to |i+1\rangle$, open boundary.

`chain_sites(Lx, Ly, ell)`

Returns the chain columns `np.arange(x0, x0 + ell)` with $x_0 = (N_x - \ell)//2$, and the chain row $N_y//2$; as `chain_x`, `chain_row` in Julia.

`ham(Lx, Ly, ell, S=S_z, t=1.0, mu=-7.0, alpha=0.9, Delta=2.4, J=5.8)`

```
Tx = sp.kron(sp.kron(Iy, shift(Nx)), hx)
Ty = sp.kron(sp.kron(shift(Ny), Ix), hy)
H = sp.kron(sp.identity(N), -mu * tzs0 + Delta * txs0) + Tx + Tx.conj().T + Ty + Ty.conj().T
```

Sparse $4N \times 4N$ BdG matrix (CSR) in the basis $4n+c$, $n = i_y N_x + i_x$. With this index a site operator is `kron(A_y, B_x)` and the full matrix `kron(site operator, 4×4 spinor matrix)`. On-site $-\mu\tau_z + \Delta\tau_x$; x bonds `kron(I_y, shift_x)`$\otimes h_x$, which are open within each row (no wrap to the next row, unlike the Julia register shift, so no mask is needed); y bonds `kron(shift_y, I_x)`$\otimes h_y$; $h_x = -t\tau_z + i\alpha\tau_z\sigma_y$, $h_y = -t\tau_z - i\alpha\tau_z\sigma_x$; both plus their conjugate transposes. The exchange adds `kron(diag(v), t0s_k)` with $v_n = JS_k(i_x)$ on the chain sites, skipped when $\max|v| \le 10^{-14}$. Used by every section.

3.3 Section 2: check against the tensor-network Hamiltonian

`julia_to_python(Hj, N)`

```
W[4*n + 0, 0*N + n] = 1            # c_up      <- (nb = 0, up)
W[4*n + 1, 1*N + n] = 1            # c_dn      <- (nb = 0, dn)
W[4*n + 2, 2*N + 1*N + n] = 1      # c_dag_dn  <- (nb = 1, dn)
W[4*n + 3, 2*N + 0*N + n] = -1     # -c_dag_up <- (nb = 1, up)
```

Converts a dense Julia matrix (index $n_b \cdot 2N + \sigma N + n$) to the Python basis: $WH_{\mathrm{TN}}W^T$ with the signed permutation W (comments in the listing transliterated). The minus sign is the $-c_\uparrow^\dagger$ of kwant's spinor.

Check. For every key of `data/tn_check_H.npz` the cell parses L_x, L_y, ℓ and the texture from the key, builds `ham(...)` with `S_z` or `S_tilt`, and prints $\max|WH_{\mathrm{TN}}W^T - H_{\mathrm{py}}|$ element by element; it then asserts that the largest difference is below 10^{-10}. Printed: largest difference 9.8×10^{-14}.

3.4 Section 3: lowest levels against substrate width

$L_x = 6$, $\ell = 48$, `S_z`, $L_y = 0 \ldots 6$: `eigsh(H, k=6, sigma=0, which="LM", return_eigenvectors=False)` (shift-invert with SciPy's default factorization). The sorted $|E|$ give the pair, `E[0]`, and the gap, `E[2]` (pairs of $\pm E$ appear twice). Run in the kernel (largest $D = 16\,384$). The purpose stated in the markdown: the substrate at $\mu = -7$ is gapped, so the gap converges after a few rows, and the converged values are a correctness check at sizes where dense diagonalization is no longer possible. The gap is 0.2563 for one row, 0.4283 and 0.5613 for 2 and 4 rows, and converged at 0.5491 from 8 rows ($L_y = 3$) on; the pair $|E|$ is 2.6×10^{-11} for one row and $\sim 10^{-14}$ with a substrate.

3.5 Section 4: sparse matrix size against system size

`size_info(Lx, Ly, ell)`

Builds `ham` and returns $(D, \mathrm{nnz}, \mathrm{CSR\ MB}, t_{\mathrm{build}})$, with the CSR memory `data + indices + indptr` bytes. Run in a forked process for each size of the width scan $(6, L_y, 48)$ and the square scan $(L, L, 3 \cdot 2^L/4)$, $L = 4 \ldots 10$ (the TN scan goes to $L = 11$). At 1024×1024: 8.38×10^7 nonzeros, 1693 MB in CSR, built in 1.5 s.

3.6 Section 5: zero-mode benchmark, dense and sparse

Settings: `RUN_ZM = True`; `DENSE_MAX_D = 16384` (dense only up to 64×64 sites); `zm_file = "py_zero_modes.csv"`; `zm_scan` as in Julia without 2048×2048 (13 sizes, increasing N); $\ell = 12$, `S_z`.

`pair_values(E, V)`

From a few eigenpairs around zero: the two smallest $|E|$ are the pair, and `ip` is the one with $E > 0$. Returns $E_f = E_{ip}$, $\Delta Q = \sum_n (|v_{n0}|^2 + |v_{n1}|^2) - (|v_{n2}|^2 + |v_{n3}|^2) = \langle\psi_f|\tau_z|\psi_f\rangle$ (eigenvectors are normalized), and the gap, the third smallest $|E|$.

`solve(Lx, Ly, method, ell=12)`

Zero modes of one system by one method; returns (eigensolver time, E_f, ΔQ, gap). The time includes the factorization but not the build of H.

- `"superlu"`: `splu(H.tocsc(), permc_spec="MMD_AT_PLUS_A")`, then `eigsh(H, k=4, sigma=0, which="LM", OPinv=LinearOperator(..., matvec=lu.solve))`, i.e. Lanczos on H^{-1}. The ordering is the minimum-degree ordering of $A^T + A$, suited to the structurally symmetric matrix; the markdown states that it halves the fill compared with `eigsh`'s default (COLAMD) and makes the solve 2.3–2.5 times faster (256^2: 21 s to 9 s; 512^2: 276 s to 110 s). SuperLU reports a failed allocation by printing to stderr and raising an empty `MemoryError`; `solve` re-raises it as `MemoryError("SuperLU: not enough memory to perform factorization")` so the status in the CSV is informative. At 1024×1024 this happens within a second with a few GB in use on a 188 GB machine (SuperLU's limits, not the machine's), and that size is recorded as failed.
- `"mumps"`: `mumps.Context().factor(H.tocoo(), ordering="metis")` (analysis and numerical factorization with METIS nested dissection), then the same `eigsh` with `ctx.solve`. Per the markdown, METIS gave the smallest factorization at 256^2 (2.6 s, against 3.1 s for SCOTCH and 3.2 s for AMD), and MUMPS's 64-bit indices let it factorize 1024×1024.
- `"mumps_ftz"`: the same as `"mumps"` with the flush-to-zero and denormals-are-zero bits (FTZ, bit 15; DAZ, bit 6) of the x86 MXCSR register set for the factorization and the solves, so that subnormal numbers are treated as zero. Both MUMPS methods call `ftz_lib.ftz_daz(on)` (on for `mumps_ftz`, off for `mumps`) before the timer and `ftz_lib.restore_csr` in a `finally`, so the kernel's MXCSR is unchanged after the warm-up `solve` that runs in the kernel. `ftz_lib` is `msh_substrate/ftz.c` (two functions, `ftz_daz(on)` returning the previous MXCSR and `restore_csr(c)`), compiled with `cc -O2 -shared -fPIC` on first use into `data/libftz.so` and loaded with `ctypes`. `MUMPS_METHODS = ("mumps", "mumps_ftz")`.
- `"dense"`: `la.eigh(H.toarray(), subset_by_index=[2N-2, 2N+1])`, the four eigenpairs in the middle of the spectrum, about twice as fast as the full spectrum; the $O(D^3)$ tridiagonal reduction dominates either way.

Driver and resumable CSV. Columns `Lx, Ly, ell, N, D, method, t, rss_MB, E_f, dQ, E_gap, status`. `done` is the set of $(L_x, L_y, \text{method})$ already in the file. If `RUN_ZM`: one warm-up `solve(4, 0, m)` per method in the kernel; the baseline peak of a forked process is printed (`in_fresh_process(lambda: None)`); the header is written if nothing is done; then for each size and method (dense only while $D \leq$ `DENSE_MAX_D`) a missing row is computed with `in_fresh_process(solve, ...)` and appended at once. A failed or killed run is recorded with NaN values and its status. Finally the file is read, sorted by (N, method) and printed. `RUN_ZM = False` only loads it; deleting the file starts over. MUMPS at 1024×1024: 1013 s and 28.6 GB, with FTZ/DAZ 236 s and the same memory (explained in Part III, section 5). The `mumps_ftz` rows were appended later from a kernel that had run only the setup cells, so their baseline is about 80 MB instead of 110 MB; dense at 64×64: 886 s and 4.4 GB; SuperLU at 1024×1024: `MemoryError: SuperLU: not enough memory to perform factorization`.

3.7 Section 6: MUMPS estimates beyond the benchmark

`mumps_estimate(L)`

Builds `ham(L, L, 12)` (COO), runs only MUMPS's analysis step (`ctx.analyze(H, ordering="metis")`: ordering and symbolic factorization, no numerical work) and returns $(L, L, N, D,$ `est_nonzeros`, `est_flops`, `est_mem_incore`, `est_mem_ooc`, $t_{\text{analysis}})$ from `ctx.analysis_stats`: the predicted number of entries in the factors, the floating-point operations of the factorization, and the in-core and out-of-core memory in MB.

`RUN_EST = True` (or a missing file) runs it in a forked process for $L = 7 \dots 11$ and rewrites `py_mumps_estimates.csv`; then the table is printed. The markdown states that the in-core estimate is 7 % (512^2) and 2.5 % (1024^2) below the measured peak of section 5, so the 2048×2048 value (about 120 GB; roughly two hours at the speed measured at 1024^2) is reliable. The estimates depend only on the sparsity pattern, which is the same at both parameter points with `S_z`. At 2048×2048: 5.76×10^9 factor entries, 1.54×10^{13} flops, 121184 MB in core, 14703 MB out of core.

3.8 Section 7: charge difference, mask method and single-mode formula

The CZ, QFT and period-finding notebooks and `msh_chain_python_ed.ipynb` compute the charge difference from the many-body densities of the two logical states (the *mask method*); this notebook and the TN code use the single vector ψ_f. The section shows that they are the same quantity.

Mask method. M is the full $4N \times 4N$ unitary of eigenvectors sorted by energy: column $2N$ is ψ_f $(+E_f)$, column $2N-1$ its partner $C\psi_f$ $(-E_f)$, columns $0 \dots 2N-1$ the negative-energy states. Logical state $|0\rangle$ fills columns $0 \dots 2N-1$; $|1\rangle$ swaps column $2N-1$ for column $2N$. With $G_j = M_{:,\text{mask}_j} M^{\dagger}_{:,\text{mask}_j}$,

$$\rho_j(\mathbf{r}) = \sum_{\sigma} (G_j)_{\mathbf{r}\sigma,\mathbf{r}\sigma}, \qquad \delta n = \rho_1 - \rho_0, \qquad \Delta Q = \sum_{\mathbf{r}} \delta n(\mathbf{r}).$$

Reduction to one vector. The two masks share every column but one, so

$$G_1 - G_0 = |\psi_f\rangle\langle\psi_f| - |C\psi_f\rangle\langle C\psi_f|.$$

The particle part of $C\psi_f$ is the complex conjugate of the hole part v of ψ_f up to a spin rotation $(C = \tau_y \sigma_y K)$, so the particle diagonal is $\delta n(\mathbf{r}) = \sum_\sigma |u_\sigma(\mathbf{r})|^2 - \sum_\sigma |v_\sigma(\mathbf{r})|^2$ and $\Delta Q = \langle\psi_f|\tau_z|\psi_f\rangle$. The bulk drops out of the difference; the identity is exact, also during a ramp (all columns evolve with the same unitary) and column by column for several modes. The mask method needs all of M (dense `eigh`, $O(D^3)$ time, $O(D^2)$ memory; per the markdown 64×64 sites take 4.3 GB and 886 s, and 1024×1024 would need about 2.8×10^5 GB); its extra output, the absolute densities ρ_0, ρ_1, is the bulk charge, which cancels in every compared quantity. Hence only the single-mode formula is used for the scaling study. Conventions that must match: ψ_f is the $+E_f$ member (the partner gives $-\Delta Q$), and the density counts electrons, the components $(c_\uparrow, c_\downarrow)$.

`mask_check(Lx, Ly, ell=12)`

```
masks = np.zeros((2, 4*N), bool)
masks[1, 2*N] = True                                  # |1>: psi_f filled
masks[:, :2*N] = np.flip(~masks[:, 2*N:], axis=1)     # column 2N-1-k = C(column 2N+k)
```

One full dense `la.eigh` (ascending energies, timed). The masks are built as in `msh_chain_python_ed.ipynb`: the upper half says which positive-energy columns are filled, and the lower half is its flipped complement, because column $2N - 1 - k$ is the partner of column $2N + k$. For each mask ρ_j is the particle diagonal of G_j, computed as the row sums of $|M_{:,\text{mask}}|^2$ over the particle components. The single-mode density uses column $2N$ alone. $C\psi_f$ is computed as

`kron(sy, sy)` applied to the conjugated per-site spinors ($C = \tau_y \sigma_y K$) and compared with column $2N - 1$. Returns a dict with N, D, E_f, the `eigh` time, the overlap $|\langle M_{:,2N-1} | C\psi_f \rangle|$, ΔQ from both methods, $\max |\delta n_{\text{mask}} - \delta n_{\text{single}}|$ and both densities as $N_y \times N_x$ arrays.

Run in forked processes for 64×1, 64×4, 16×16, 32×32 (`mask_scan`); the cell prints one line per system and plots $\delta n(i_x, i_y)$ of the 32×32 system (mask method) and $|\delta n_{\text{mask}} - \delta n_{\text{single}}|$. Overlap 1.000000000000 in all four cases, $|\Delta Q_{\text{mask}} - \Delta Q_{\text{single}}| \leq 2.1 \times 10^{-15}$, δn differences $\leq 2.0 \times 10^{-12}$.

3.9 Section 8: a substrate the zero modes leak into

8a. $\mu \times J$ scan for three values of Δ

Fixed geometry 64×32 ($L_x = 6$, $L_y = 5$), $\ell = 32$ in row 16. Module-level masks over the (i_y, i_x) grid: `ends_s`, within 3 columns of either chain end (all rows); `mid_s`, the middle third of the chain columns; `dy_s` $= |i_y - i_{y0}|$.

`scan_point(args)`

For `args = (Delta, mu, J)`: `eigsh(ham(...), k=6, sigma=0, which="LM")`, levels sorted by $|E|$. With ρ the total weight (all four components) per site of the lowest state, returns $(\Delta, \mu, J, E_0, E_2, E_4$, `w_ends`, `w_mid`, `w_row`, `rms_dy`, `w_far`): the first three distinct levels $|E|$ (indices 0, 2, 4 of the $\pm$ pairs), the weight near the chain ends, in the middle third, on the chain row, the rms distance $\sqrt{\sum \rho \, \Delta y^2}$ from the chain row, and the weight four or more rows away.

Grid: $\Delta \in \{2.4, 1.2, 0.6\}$, $\mu = -7 \ldots 0$ in steps of 0.25, $J = 1 \ldots 8$ in steps of 0.5: $3 \times 29 \times 15 = 1305$ points, `Pool(8).map(..., chunksize=4)`, written to `py_mu_scan.csv` (`RUN_MU = False` loads it). **Topological criterion:** $E_0/E_2 < 0.05$ (a pair well below the next level) and `w_ends` > 0.5 (the lowest state sits at the chain ends). Figure: top row $\log_{10}(E_0/E_2)$ per Δ with the topological region outlined in red; bottom row `rms_dy` inside it; dashed line at the band bottom $\mu = -4.741$. Printed: the number of topological points per Δ, the topological point with the largest E_2 below and inside the band, and four selected points, including $(0.6, -3, 3)$. 1305 points in 198 s; topological: 97 ($\Delta = 2.4$), 113 (1.2), 90 (0.6).

8b. The point $\Delta = 0.6$, $\mu = -3$, $J = 3$

`pair_levels(Lx, Ly, ell)`

`eigsh` (6 levels) at `PC = dict(mu=-3.0, Delta=0.6, J=3.0)`. Returns $|E_0|$, $|E_2|$, ΔQ of the $+E$ member of the pair, and the weight per site of that state as an $N_y \times N_x$ array.

Three checks, each size in a forked process: (1) splitting against chain length, $\ell = 16 \ldots 128$ on 256×32, printing E_0, E_2, E_0/E_2, ΔQ; (2) convergence with substrate height, $N_x = 256$, $\ell = 64$, $N_y = 8 \ldots 128$; (3) the weight per row, averaged over $\pm \Delta y$, at $\Delta y = 0, 1, 2, 4, \ldots, 48$ on 256×128. These produce the numbers quoted in the model description (decay length about 7 sites, $\ell = 64$, $e^{-|\Delta y|/2}$ decay). $E_0/E_2 = 1.0 \times 10^{-4}$ at $\ell = 64$ (256×32); E_0, gap and ΔQ converge from $N_y = 64$ on ($E_0 = 9.0477 \times 10^{-6}$, gap 0.09025, $\Delta Q = 4.5834 \times 10^{-4}$); the row weight falls from 0.32 on the chain row to 3.6×10^{-3} at $\Delta y = 8$ and 1.4×10^{-11} at 48.

8c. MUMPS reference at the TN sizes

`solve_candidate(Lx, Ly, ell=64)`

`solve(..., "mumps")` at `PC`: METIS factorization and shift-invert `eigsh(k=4)`; returns (time including the factorization, E_f, ΔQ, gap) via `pair_values`.

`cand_scan` $= (7,6),(8,8),(9,9),(10,10)$, i.e. the TN sizes up to 1024×1024. Same resumable logic as section 5 with `py_candidate.csv` (columns `Lx, Ly, ell, N, t, rss_MB, E_f, dQ, E_gap, status`); there is no `RUN` flag, missing sizes are always run. The factorization depends only on the sparsity pattern, so the estimates of section 6 hold; the cell prints the table and a 2048×2048 line with time $t_{1024} \times \text{flops}_{2048}/\text{flops}_{1024}$ and the in-core memory estimate, both marked "~". MUMPS: 1.0 s (128×64), 10.7 s (256^2), 53.7 s (512^2), 295.1 s (1024^2); estimate at 2048^2: ~2326 s and ~121 GB.

4 Part III — Scaling notebook (`msh_substrate_scaling.ipynb`)

Loads the CSV files of the other two notebooks and compares the cost of the methods; it computes nothing expensive. Inputs: `tn_zero_modes.csv`, `py_zero_modes.csv`, `py_mumps_estimates.csv`, and in section 6 `tn_candidate.csv`, `py_candidate.csv`.

4.1 Loading and helpers (cell 1)

`tn` is the TN table sorted by N; `py` the Python table (with string columns); `est` the MUMPS estimates. Rows of `py` whose status is not `"ok"` are printed as failed and left out of `M`, the dictionary of successful rows per method (`superlu`, `mumps`, `mumps_ftz`, `dense`), sorted by N. `mu` and `mf` map sizes to the MUMPS rows without and with FTZ/DAZ.

`key(r), ref(a, col)`

`key` returns (L_x, L_y) of a row. `REF` maps sizes to the exact reference row: SuperLU rows, overwritten by MUMPS rows where MUMPS ran (i.e. MUMPS wherever available). `ref(a, col)` returns the reference value of column `col` for each row of `a`, NaN where there is none (2048×2048).

`scan(a, which)`

Rows of the width scan (`Lx == 6`) or the square scan (`Lx == Ly`), sorted by N. 64×64 belongs to both.

`fit(N, t, npts)`

Power law $t = cN^p$ by least squares in $\log t$ against $\log N$ through the `npts` largest finite points; returns (p, c, N_{fit}).

MUMPS at 2048×2048. `t_mumps_2048` $= t_{\text{MUMPS}}(1024^2) \cdot \text{flops}_{2048}/\text{flops}_{1024}$, the measured 1024×1024 time scaled by the ratio of the estimated factorization flops. The cell prints the number of sizes per method and the estimate in GB and hours. `t_mumps_ftz_2048` is the same from the FTZ/DAZ time. Printed: 121 GB in core, ~7983 s (2.2 h), with FTZ/DAZ ~1859 s.

4.2 Section 1: time

Two log–log panels (width scan, square scan) of the wall time against N: TN `t_total` (build of H and K, three DMRGs, pairing), and the eigensolver time (including the factorization) of SuperLU, MUMPS, MUMPS (FTZ/DAZ) and dense. On the square panel, dashed power-law fits through the largest points (TN and both MUMPS series four, SuperLU and dense three) with the exponent in the legend, and the two MUMPS estimates as open diamonds. Square scan: TN (DMRG) $N^{0.14}$, SuperLU $N^{1.54}$, MUMPS $N^{1.51}$, MUMPS (FTZ/DAZ) $N^{1.19}$, dense $N^{3.07}$. The FTZ/DAZ exponent is lower because the subnormal slowdown grows with the size (1.6 times at 512^2, 4.3 at 1024^2).

4.3 Section 2: memory

One log–log panel of peak memory against N: the Julia process RSS (whole process up to that size, dominated by runtime and compiled code), the TN data itself (`mem_K_MB + mem_psi_MB`), the RSS of each Python method (its own forked process, baseline about 110 MB included), the MUMPS in-core estimates (open diamonds), the dense-matrix line $16D^2$ bytes, and the 188 GB of the machine. Printed: the growth exponents of the SuperLU and MUMPS RSS over their three largest square sizes (`fit`), measured against estimated MUMPS memory for $N \geq 65\,536$, and the

TN data and process RSS at 2048×2048. RSS grows as $N^{0.98}$ (SuperLU) and $N^{1.05}$ (MUMPS) over the three largest sizes; MUMPS measured/estimated: 1552/1326 MB (256^2), 6594/6117 MB (512^2), 28636/27913 MB (1024^2), i.e. the measured RSS is 17 %, 8 % and 2.6 % above the estimate; MUMPS with FTZ/DAZ within 0.7 % of MUMPS for $N \geq 65\,536$ (not plotted); TN data 1.40 MB and Julia RSS 1841 MB at 2048×2048.

4.4 Section 3: accuracy

$|E_f - E_f^{\text{exact}}|$ and $|\Delta Q - \Delta Q^{\text{exact}}|$ against N for TN, SuperLU, MUMPS (FTZ/DAZ) and dense, with `ref` as the exact value; differences are clipped at 10^{-17} for the log scale (so SuperLU at sizes where it is its own reference shows at the floor). The TN values use `cutoff_K = 1e-20`. Printed: the largest TN errors over the sizes with a reference, the largest relative differences of MUMPS with FTZ/DAZ from MUMPS (1.9×10^{-12} in E_f, 1.9×10^{-11} in ΔQ, 3.0×10^{-13} in the gap), the MUMPS values at 1024×1024, and the TN values at 2048×2048 for comparison with the converged ones. Largest TN errors 2.5×10^{-8} (ΔQ) and 1.1×10^{-11} (E_f); MUMPS at 1024^2: $E_f = 3.7122847509 \times 10^{-4}$, $\Delta Q = -3.4466003632 \times 10^{-3}$; TN at 2048^2: $3.7122846056 \times 10^{-4}$, $-3.4465999587 \times 10^{-3}$.

4.5 Section 4: bond dimensions

$\chi(H)$, $\chi(K)$ and the largest $\chi(\psi)$ against N (semilog). The point of the plot, per the markdown: if they stay bounded, the TN cost grows only through the number of qubits, $\log_2 N + 2$. Both stay bounded: $\chi(H) = 7$, $\chi(K) = 31$–38, $\chi(\psi) \leq 26$ at every size.

4.6 Section 5: summary and crossover

A table per TN size: times of TN, SuperLU, MUMPS, MUMPS with FTZ/DAZ (`MU FTZ`), dense, the ratios MUMPS/TN and MUMPS (FTZ/DAZ)/TN (above 1: TN faster), and the four peak memories (TN, SuperLU, MUMPS, dense). The helper `cell(m, k, col, fmt, w)` formats one entry: a dash if the method has no row for that size, `failed` if its status is not `"ok"`. At 2048×2048 the MUMPS columns hold the estimates, marked ~. MUMPS/TN = 0.59, 8.05 and FTZ/TN = 0.36, 1.88 at 512^2, 1024^2; at 2048^2 (estimates) 56.9 and 13.3.

Crossover. For each MUMPS series, on the square scan, the points $(N, \log(t_{\text{MUMPS}}/t_{\text{TN}}))$ at the sizes where both ran (measured MUMPS only); between the first pair of consecutive sizes where the log-ratio changes from negative to non-negative, N_c is found by linear interpolation of the log-ratio in $\log N$, and printed with $\sqrt{N_c}$ and the bracketing sizes. Printed: MUMPS $N_c \approx 3.47 \times 10^5$ sites ($\approx 589 \times 589$), MUMPS (FTZ/DAZ) $N_c \approx 6.17 \times 10^5$ ($\approx 786 \times 786$), both between 512^2 and 1024^2.

MUMPS and subnormal numbers (markdown). Without flush-to-zero MUMPS needs 1013 s at 1024×1024, 14.5 times its 512×512 time where $N^{1.5}$ would give about 8, and three times as long as at the second point (308 s) with the same fill and the same peak memory (28.6 GB). The cause is subnormal numbers: at $\mu = -7$ the Green's function decays within a fraction of a site, factor entries underflow towards 10^{-308}, and arithmetic on subnormals is slow on x86. A table of factorization times alone (separate runs) shows it: at $\mu = -7$ 29.8 s / 14.8 s (512^2, default / FTZ/DAZ) and 770.5 s / 106.6 s (1024^2); at the second point 14.4 s / 14.4 s and 112.4 s / 107.7 s, so with FTZ/DAZ both points cost the same, as the fill says they should. The results do not change (E_f, ΔQ, gap agree to 2×10^{-11} relative at every size, at 512^2 in every printed digit). The markdown calls the FTZ/DAZ series the cost of the algorithm and its crossover ($\approx 786^2$) the fair one; the default series is MUMPS out of the box. The 2048^2 estimate falls from ~8000 s to ~1900 s, still 13 times the TN time.

4.7 Section 6: a substrate the zero modes leak into

Loads `tn_candidate.csv` and `py_candidate.csv` (successful rows only), sorted by N. The MUMPS time at 2048×2048 is estimated as above from the measured 1024^2 time at this point (`t_mc_2048`); the memory estimate is the same as at the benchmark point (same pattern). The exact values `ex = (E_f = 9.04770870e-06, dQ = 4.5834351e-04)` (converged MUMPS values of Python section 8c) are hard-coded. A table per TN size: TN and MUMPS times and their ratio, TN process RSS and MUMPS RSS (estimates marked ~), χ_ψ, and the TN errors $|E_f - \text{exact}|$, $|\Delta Q - \text{exact}|$. Two log–log panels (time, peak memory) show both methods at this point, the MUMPS estimate as a dotted segment from 1024^2 to 2048^2, and the square-scan curves of the benchmark point as open symbols (in the time panel also MUMPS with FTZ/DAZ). The second point has no subnormal slowdown (factorization 107.7 s with FTZ/DAZ against 112.4 s without at 1024^2), so its MUMPS times are not rerun with FTZ/DAZ. The crossover is computed as in section 5, here including the 2048^2 estimate; if MUMPS is faster at every size the cell says so. Note that the TN RSS is the process peak including section 7, while the MUMPS RSS is per forked process. MUMPS/TN $= 0.00, 0.01, 0.03, 0.10, 0.91$ from 128×64 to 2048^2; TN errors $\leq 2.6 \times 10^{-12}$ (E_f) and $\leq 9.0 \times 10^{-10}$ (ΔQ); printed: no crossover up to 2048×2048 (MUMPS is faster at every size, with its estimate at 2048^2), but the TN needs 2.3 GB there against ~121 GB.